

Task 1

Hardware Property	Local Measured Value
CPU Model & Architecture	Intel Core i5-12500H, x86-64, Hybrid Architecture
Base Clock	2.50 GHz
Max Boost Clock	Up to 4.50 GHz
Physical Hardware Cores	12 — 4 P-cores + 8 E-cores
Logical Cores (SMT/Threads)	16
L1 Data / Instruction Cache	L1d: 48 KB/core (P), 32 KB/core (E) L1i: 32 KB/core (P), 64 KB/core (E)
L2 Cache (Per Core / Total)	9 MB total
L3 Shared Last-Level Cache	18 MB
SIMD Vector Extensions	AVX, AVX2, FMA; AVX-512 not supported

Q1.1

The Intel Core i5-12500H demonstrates different Flynn's Taxonomy execution modes depending on the workload.

SISD (Single Instruction, Single Data): A single-threaded sequential program can execute one instruction stream on one CPU core. For example, a single-thread compiler pass can operate sequentially on one P-core.

SIMD (Single Instruction, Multiple Data): The processor supports AVX and AVX2 vector extensions. These allow a single instruction to operate on multiple data elements simultaneously. This is useful for vector operations, numerical calculations, and matrix computations.

MIMD (Multiple Instruction, Multiple Data): The processor has 12 physical cores and 16 logical threads. Multiple independent threads can execute different instruction streams and

operate on different data simultaneously. For example, the operating system can schedule different processes and threads across the P-cores and E-cores.

Therefore, the same processor can exhibit SISD, SIMD, and MIMD behavior depending on the software workload and level of parallelism.

Task 2

Threads / Processes (N)	Run 1 (s)	Run 2 (s)	Run 3 (s)	Avg. Time T_{avg} (s)	Speedup S_{avg}	Efficiency E_{avg}
1	42.040	35.865	31.561	36.489	1.00×	100.00%
2	19.140	20.530	20.047	19.906	1.83×	91.67%
4	14.597	12.206	12.343	13.049	2.80×	70.11%
8	10.881	10.530	11.900	11.103	3.29×	41.33%
16	8.447	9.551	9.787	9.262	3.94×	24.62%
32	9.696	10.105	9.746	9.849	3.70×	11.55%

Q2.1

The benchmark shows strong scaling from 1 to 16 worker processes, but the scaling becomes saturated before 16 workers and reverses at 32 workers. The best measured performance occurs at 16 workers, with an average execution time of 9.262 seconds and a speedup of 3.94× relative to the 1-worker baseline. Increasing the worker count to 32 increases the average execution time to 9.849 seconds, reducing the speedup to 3.70×. Therefore, the exact point where scaling reverses is between 16 and 32 workers.

This behavior is consistent with the hardware topology of the Intel Core i5-12500H, which has 12 physical cores and 16 logical processors. Up to 12 physical cores, additional workers can obtain more physical CPU execution resources. The processor has 16 logical processors because the four P-cores support Hyper-Threading, while the eight E-cores provide one hardware thread each. Once the workload reaches 16 workers, all logical processors can be utilized.

At 32 workers, there are twice as many software processes as logical processors. The operating system must therefore time-slice and schedule multiple processes on the same logical processors. This introduces additional scheduling and process-management overhead. Cache and memory contention can also increase because more processes compete for shared CPU resources. These effects explain why adding workers beyond

the 16 logical processors does not improve performance and instead slightly increases execution time.

The benchmark therefore demonstrates diminishing returns from parallelization: speedup increases from 1.00× at one worker to 3.94× at 16 workers, but decreases to 3.70× at 32 workers. The results indicate that the contention wall is reached around the hardware limit of 16 logical processors, with oversubscription becoming visible at 32 workers.

Task 3

Run	Measured Output	Error (10^7 – Actual)
#1	1,000,224	8,999,776
#2	1,001,229	8,998,771
#3	1,000,177	8,999,823
#4	1,000,304	8,999,696
#5	1,000,283	8,999,717
#6	1,000,346	8,999,654
#7	1,000,220	8,999,780
#8	1,000,283	8,999,717
#9	1,000,251	8,999,749
#10	1,000,126	8,999,874

Q3.1

The shared counter is subject to a Read-Modify-Write (RMW) race condition.

Conceptually, `counter += 1` consists of three steps: Load the current value, Add 1, and Store the result. These operations are not guaranteed to occur as one indivisible operation. If two threads read the same value before either stores its result, both calculate the same next value. When they store their results, one update overwrites the other, causing a lost update.

For example, if the counter is 100, Thread A and Thread B can both load 100. Both calculate 101, and both subsequently store 101. Although two increments were

requested, the counter increases only from 100 to 101. Repeated millions of times, these lost updates produce a measured value significantly below the theoretical 10,000,000.

Q3.2

Wrapping the counter operation in a mutex prevents multiple threads from modifying the shared variable simultaneously. The lock ensures that the Load → Add → Store sequence is performed by only one thread at a time. As a result, the locked implementation produces the correct value of exactly 10,000,000.

Synchronization normally introduces overhead because threads must acquire and release the lock, and competing threads may have to wait for the lock to become available. This reduces parallelism and can cause contention. However, in this experiment the unlocked version intentionally uses `time.sleep(0)` during every increment to force thread scheduling and expose the race condition. Therefore, its much larger execution time should not be interpreted as evidence that locks are inherently faster. The timing primarily reflects the artificial scheduling overhead introduced to make the race condition reproducible.

Task 4

Question	Answer
T1T_1	36.489 s
T2T_2	19.906 s
Parallel fraction pp	0.909
Sequential fraction 1-p1-p	0.091
Amdahl SmaxS_{max}	11.0×
Amdahl S64S_{64}	8.60×
Gustafson S64S_{64}	58.27×

Q4.1

Amdahl's Law describes **strong scaling**, where the problem size remains fixed while the number of processors increases. The sequential part of the algorithm cannot be parallelized, so it becomes a bottleneck as more cores are added. For this workload, the estimated sequential

fraction is about 9.1%, which limits the theoretical speedup to approximately 11×, even with an unlimited number of cores.

Gustafson's Law describes **weak scaling**, where the problem size increases as more processors are added. With 64 cores, more of the total work can be assigned to the parallel portion, while the sequential portion remains approximately constant. Therefore, Gustafson's model predicts a much larger speedup of about 58.27×.

Thus, the two models answer different questions: **Amdahl asks how much faster the same fixed-size problem can become, while Gustafson asks how much larger a problem can be handled in approximately the same amount of time when more cores are available.** This is why Amdahl predicts a hard bottleneck while Gustafson can predict near-linear scaling for a sufficiently parallel workload.